

# Otimização de Consultas em Bancos de Dados

---



Eduardo Ogasawara

[eogasawara@cefet-rj.br](mailto:eogasawara@cefet-rj.br) | <https://eic.cefet-rj.br/~eogasawara>



A otimização de consultas é um processo fundamental em SGBDs que visa melhorar o desempenho e a eficiência da execução de consultas SQL. O otimizador analisa diferentes estratégias de execução — por meio de otimização lógica (transformações algébricas) e física (recursos reais do sistema) — selecionando aquela que minimiza o custo computacional, reduzindo tempo de resposta e consumo de recursos.

---

## Otimização Lógica

- Transforma expressões algébricas em formas equivalentes mais eficientes
- Opera com regras de equivalência da álgebra relacional
- Independente de algoritmos e custos físicos

---

## Otimização Física

- Escolhe algoritmos concretos de avaliação
- Considera índices, métodos de junção e ordenação
- Baseia decisões em custos estimados reais

---

## Definição

Representação formal de consultas: cada nó é uma operação relacional e cada folha é uma relação base.

---

## Ordem de Execução

Avaliação de baixo para cima, combinando resultados intermediários. Árvores equivalentes podem ter custos distintos.

---

## Estrutura

Folhas são tabelas de entrada; nós internos são operações (seleção, projeção, junção); a raiz é o resultado final.

---

## Papel na Otimização

Base para otimização lógica: regras de equivalência reorganizam operações para reduzir o custo de execução.

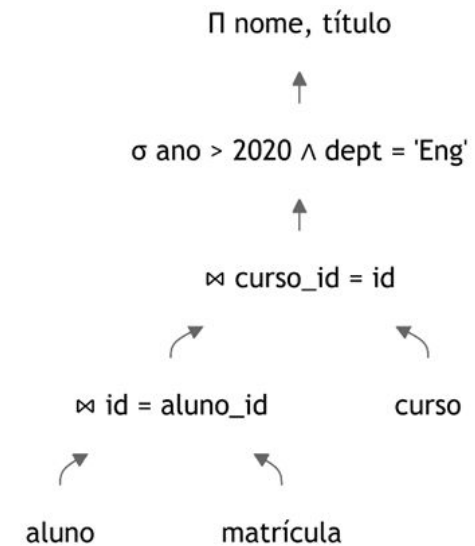
Como uma consulta SQL é transformada em uma árvore de processamento usando álgebra relacional.

## Esquema Relacional

- aluno(id, nome)
- matrícula(aluno\_id, curso\_id, ano)
- curso(id, título, dept)

## Objetivo da Consulta

Obter os nomes dos alunos e os títulos dos cursos do departamento de Engenharia, iniciados após o ano de 2020.



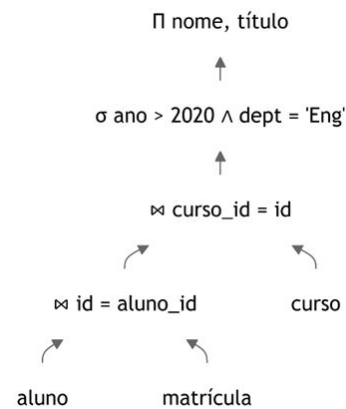
## Expressão em Álgebra Relacional

$$\pi_{nome, titulo} \left( \sigma_{ano > 2020 \wedge dept = 'Eng'} \left( (aluno \bowtie_{id = aluno\_id} matricula) \bowtie_{curso\_id = id} curso \right) \right)$$

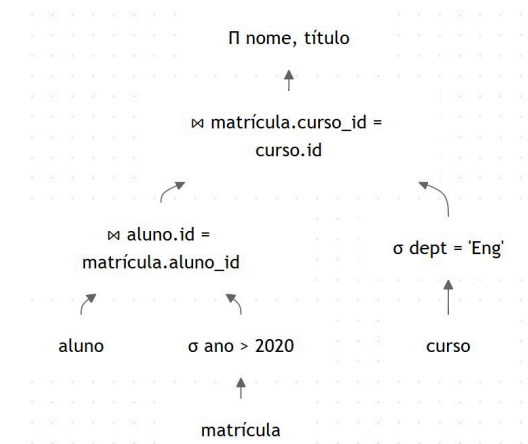
A otimização de consultas transforma a árvore inicial aplicando técnicas como push-down de seleções, que empurra os filtros para mais perto das relações base, reduzindo o tamanho dos resultados intermediários.

```
SELECT a.nome, c.título
FROM aluno AS a
JOIN matrícula AS m ON a.id = m.aluno_id
JOIN curso AS c ON m.curso_id = c.id
WHERE m.ano > 2020 AND c.dept = 'Eng';
```

**Antes:** Filtro aplicado tarde, após todas as junções serem executadas.



**Depois:** Predicados decompostos e empurrados para as relações apropriadas antes das junções.



**Álgebra Relacional Otimizada:**

$$\pi_{nome, título} \left( (aluno \bowtie_{id=aluno\_id} \sigma_{ano>2020}(matrícula)) \bowtie_{curso\_id=id} \sigma_{dept='Eng'}(curso) \right)$$

❏ **Observação:** A árvore otimizada é equivalente à original, mas mais eficiente. Ela reduz o tamanho dos resultados intermediários antes das junções.

A otimização baseada em custo avalia numericamente diferentes planos de execução para selecionar o mais eficiente, combinando teoria e prática para decisões fundamentadas.

---

### **Geração de Expressões Equivalentes**

Aplica regras de equivalência da álgebra relacional para criar múltiplas versões logicamente equivalentes da consulta original.

---

### **Estimativa de Custos**

Calcula o custo estimado de cada plano usando estatísticas do catálogo e modelos de custo para I/O, CPU e memória.

---

### **Anotação de Expressões**

Cada expressão é anotada com algoritmos de acesso, métodos de junção e outras escolhas físicas, gerando planos alternativos.

---

### **Seleção do Plano Ótimo**

Escolhe o plano de menor custo estimado para execução, equilibrando precisão das estimativas com tempo de otimização.

O otimizador depende de estatísticas do catálogo para tomar decisões informadas sobre planos de execução.

---

### Cardinalidade das Relações

Número total de tuplas em cada relação, essencial para estimar o tamanho dos resultados.

---

### Blocos de Armazenamento

Quantidade de blocos físicos no disco, usada para estimar custos de I/O.

---

### Valores Distintos

Número de valores únicos por atributo, fundamental para estimar a seletividade de predicados.



**Aplicação:** Estatísticas desatualizadas podem levar o otimizador a escolhas subótimas de junções, índices e métodos de acesso.

## **Equivalência em Conjuntos**

Duas expressões são equivalentes se geram o mesmo conjunto de tuplas para qualquer instância do banco de dados. A ordem das tuplas é irrelevante.

## **Equivalência em Multiconjuntos**

Em SQL, duas expressões são equivalentes se geram o mesmo multiconjunto de tuplas, preservando duplicatas — distinção essencial para otimização prática.

As regras de equivalência da álgebra relacional são o alicerce da otimização lógica de consultas.

---

### Comutatividade

Operações como  $\bowtie$  e  $\cup$  podem ter seus operandos trocados sem afetar o resultado.

---

### Associatividade

Permite reagrupar operações em cadeia, mudando qual é executada primeiro — fundamental para otimizar múltiplas  $\bowtie$ .

---

### Distributividade

Operações como  $\sigma$  podem ser distribuídas sobre  $\bowtie$ , filtrando dados mais cedo no plano de execução.

---

## Comutatividade de Junção Natural

$$R \bowtie S \equiv S \bowtie R$$

A ordem dos operandos pode ser invertida sem alterar o resultado, permitindo escolher a ordem mais eficiente baseada em estatísticas.

---

## Comutatividade de Junção Theta

$$R \bowtie_{\theta} S \equiv S \bowtie_{\theta} R$$

Junções com predicados arbitrários também são comutativas, desde que o predicado seja ajustado adequadamente.

---

## Associatividade de Junções

$$(R \bowtie S) \bowtie T \equiv R \bowtie (S \bowtie T)$$

Permite explorar diferentes ordens de execução em consultas com múltiplas junções, podendo variar drasticamente em custo.

---

## Decomposição de Seleções Conjuntivas

$$\sigma_{\theta_1 \wedge \theta_2}(R) \equiv \sigma_{\theta_1}(\sigma_{\theta_2}(R))$$

Uma seleção com condição AND pode ser decomposta em seleções sequenciais, permitindo reordenar predicados para maior eficiência.

---

## Comutatividade de Seleções

$$\sigma_{\theta_1}(\sigma_{\theta_2}(R)) \equiv \sigma_{\theta_2}(\sigma_{\theta_1}(R))$$

Seleções sucessivas podem ser aplicadas em qualquer ordem, útil para avaliar primeiro os predicados mais seletivos.

---

## Decomposição de Projeções

$$\pi_{L_1}(\pi_{L_2}(\dots(\pi_{L_n}(R))\dots)) \equiv \pi_{L_1}(R)$$

onde  $L_1 \subseteq L_2 \subseteq \dots \subseteq L_n$

Apenas a última projeção em uma sequência é necessária, desde que contenha todos os atributos requeridos, eliminando projeções intermediárias redundantes.

---

## Comutatividade

Permite trocar a ordem dos operandos sem alterar o resultado. Válida para:

- Junção natural ( $\bowtie$ )
- Junção theta ( $\bowtie\theta$ )
- União ( $\cup$ )
- Interseção ( $\cap$ )

---

## Associatividade

Permite reagrupar operações em cadeia, alterando qual par é avaliado primeiro. Crucial para:

- Múltiplas junções em consultas complexas
- Operações de conjunto encadeadas
- Otimização de ordem de execução

O *push-down* de  $\sigma$  move filtros para mais perto das relações-base, reduzindo o volume de dados nas etapas subsequentes.

---

### $\sigma$ após $\bowtie$

A seleção ocorre depois da junção, processando um volume maior de dados intermediários.

---

### Decomposição do Predicado

O predicado é analisado para identificar componentes que referenciam apenas uma relação.

---

### $\sigma$ antes da $\bowtie$

Predicados de uma única relação são empurrados para antes da junção, filtrando dados antecipadamente.

---

**Benefício chave:** Filtrar antes da  $\bowtie$  reduz o produto  $\times$  parcial, economizando I/O, memória e tempo de CPU.

---

## Seleção sobre Junção Natural

Predicado envolvendo apenas atributos de uma relação pode ser aplicado antes da junção:

$$\sigma_{\theta}(R \bowtie S) \equiv \sigma_{\theta}(R) \bowtie S$$

(se  $\theta$  usa apenas atributos de R)

---

## Seleção com Predicados Compostos

Conjunção AND com  $\theta_1$  em R e  $\theta_2$  em S permite distribuir a seleção:

$$\sigma_{\theta_1 \wedge \theta_2}(R \bowtie S) \equiv \sigma_{\theta_1}(R) \bowtie \sigma_{\theta_2}(S)$$

---

## Seleção sobre Junção Theta

Similar à junção natural, preservando o predicado de junção original:

$$\sigma_{\theta_1}(R \bowtie_{\theta_2} S) \equiv (\sigma_{\theta_1}(R)) \bowtie_{\theta_2} S$$

---

## Distribuição de Projeção sobre Junção

A projeção pode ser distribuída sobre uma junção natural quando os atributos da junção estão incluídos nos atributos projetados:

$$\pi_{L_1 \cup L_2}(R \bowtie S) \equiv \pi_{L_1}(R) \bowtie \pi_{L_2}(S)$$

Onde  $L_1$  e  $L_2$  são listas de atributos de  $R$  e  $S$  respectivamente, e incluem os atributos de junção.

---

## Operações de Conjunto

Seleção distribui sobre união, interseção e diferença:

$$\sigma_{\theta}(R \cup S) \equiv \sigma_{\theta}(R) \cup \sigma_{\theta}(S)$$

$$\sigma_{\theta}(R \cap S) \equiv \sigma_{\theta}(R) \cap \sigma_{\theta}(S)$$

$$\sigma_{\theta}(R - S) \equiv \sigma_{\theta}(R) - \sigma_{\theta}(S)$$

Projeção também distribui sobre união:

$$\pi_L(R \cup S) \equiv \pi_L(R) \cup \pi_L(S)$$

Agregações e agrupamentos requerem tratamento especial nas transformações de consulta.

---

### Push-Down de Seleção com Agregação

Seleções sobre atributos de agrupamento podem ser aplicadas antes da agregação:

$$\sigma_{\theta}(G\Gamma_F(R)) \equiv G\Gamma_F(\sigma_{\theta}(R))$$

Válido quando  $\theta$  referencia apenas atributos em G.

---

### Decomposição de Agregações

Funções como COUNT e SUM podem ser parcialmente agregadas e depois combinadas, útil para processamento paralelo.

---

### Agregação com Junção

Sob certas condições, agregações podem ser empurradas por junções, especialmente em relações muitos-para-um em relação ao agrupamento.

 O otimizador pode converter condições HAVING elegíveis em predicados WHERE, filtrando dados antes da agregação para melhor desempenho.

---

## **Descorrelação de Subconsultas**

Subconsultas correlacionadas podem ser transformadas em junções equivalentes, eliminando a necessidade de avaliar a subconsulta para cada tupla externa. EXISTS e IN podem ser convertidos em semi-junções, melhorando drasticamente o desempenho.

---

## **Eliminação de Subconsultas Redundantes**

Subconsultas que retornam resultados constantes ou são semanticamente equivalentes a expressões mais simples podem ser eliminadas pelo otimizador. Isso reduz o custo de avaliação sem alterar a semântica da consulta.

---

## **Operações de Ordenação**

ORDER BY pode ser postergado até o final do plano de execução para evitar ordenações desnecessárias. Em alguns casos, ordenação intermediária é benéfica quando há índices ordenados ou junções sort-merge disponíveis.

Considere uma consulta sobre um banco de dados acadêmico que busca informações sobre professores e disciplinas de um departamento específico.

---

## Plano Inicial

$$\sigma_{dept='Computação'}(\text{professor} \bowtie \text{ensina} \bowtie \text{disciplina})$$

Executa a junção completa de três tabelas antes de aplicar o filtro, processando um grande volume intermediário de dados desnecessários.

---

## Plano Otimizado

$$(\sigma_{dept='Computação'}(\text{professor})) \bowtie \text{ensina} \bowtie \text{disciplina}$$

Aplica o filtro antes da primeira junção (*push-down*), reduzindo drasticamente o número de tuplas envolvidas nas operações subsequentes.

- ❏ Se o departamento de Computação tiver 10% dos professores, reduzimos o volume da primeira junção em 90%, economizando I/O, memória e tempo de CPU.

Em consultas complexas, múltiplas regras de equivalência são aplicadas em sequência para alcançar o plano ótimo.

---

### Consulta Original

$$\pi_{\text{nome,título}}(\sigma_{\text{ano}>2020 \wedge \text{dept}='Eng'}((\text{aluno} \bowtie_{\text{id=aluno\_id}} \text{matrícula}) \bowtie_{\text{curso\_id=id}} \text{curso}))$$

Forma direta da consulta SQL, aplicando filtro e projeção sobre a junção completa das três tabelas.

---

### Push-Down de Seleções

$$\pi_{\text{nome,título}}((\text{aluno} \bowtie_{\text{id=aluno\_id}} \sigma_{\text{ano}>2020}(\text{matrícula})) \bowtie_{\text{curso\_id=id}} \sigma_{\text{dept}='Eng'}(\text{curso}))$$

Cada seleção é movida para a relação apropriada, filtrando dados antes das junções.

---

### Decomposição de Seleção

$$\pi_{\text{nome,título}}(\sigma_{\text{ano}>2020}(\sigma_{\text{dept}='Eng'}((\text{aluno} \bowtie_{\text{id=aluno\_id}} \text{matrícula}) \bowtie_{\text{curso\_id=id}} \text{curso}))))$$

Separamos o predicado conjuntivo em duas seleções independentes.

---

### Push-Down de Projeção

$$\pi_{\text{nome,título}}((\pi_{\text{id,nome}}(\text{aluno}) \bowtie_{\text{id=aluno\_id}} \pi_{\text{aluno\_id,curso\_id}}(\sigma_{\text{ano}>2020}(\text{matrícula}))) \bowtie_{\text{curso\_id=id}} \pi_{\text{id,título}}(\sigma_{\text{dept}='Eng'}(\text{curso})))$$

Após filtrar, projetamos apenas os atributos ainda necessários (chaves de junção e atributos do resultado), reduzindo a largura das tabelas intermediárias.

**Resultado:** O plano otimizado processa muito menos dados em cada etapa, podendo reduzir o custo de execução em ordens de magnitude.

A otimização de consultas relacionais envolve múltiplas transformações sucessivas na árvore de expressão, aplicando regras de equivalência algébrica que preservam o resultado final enquanto reorganizam operações, reordenam junções e reposicionam seleções e projeções — reduzindo significativamente o custo de execução em termos de acessos a disco, tamanho de resultados intermediários e uso de memória.

---

## Antes da Otimização

A projeção ( $\pi$ ) posicionada tardiamente processa mais atributos do que o necessário nas operações intermediárias.

$$\pi_{\text{nome}, \text{dnome}}(\sigma_{\text{departamento}='TI'}(\text{funcionarios} \bowtie_{\text{dno}=\text{dnumero}} \text{departamento}))$$

---

## Após a Otimização

Com o "push down" de projeção ( $\pi$ ), atributos desnecessários são eliminados o mais cedo possível, reduzindo o volume de dados transferidos entre operações.

$$\pi_{\text{nome}, \text{dnome}}(\sigma_{\text{departamento}='TI'}(\pi_{\text{nome}, \text{dno}, \text{departamento}}(\text{funcionarios}) \bowtie_{\text{dno}=\text{dnumero}} \pi_{\text{dnumero}, \text{dnome}}(\text{departamento})))$$

Cada relação é reduzida às colunas usadas na junção, no filtro ou no resultado antes de a junção ser executada.

A ordenação de junção (merge join) aproveita dados previamente ordenados para realizar junções de forma eficiente.

---

### **Ordenação das Relações**

Ambas as relações são ordenadas pelo atributo de junção, aproveitando índices existentes ou realizando ordenação externa.

---

### **Varredura Simultânea**

Ponteiros avançam simultaneamente pelas duas relações ordenadas, comparando valores e identificando correspondências.

---

### **Geração do Resultado**

Tuplas correspondentes são combinadas e adicionadas ao resultado, que já emerge naturalmente ordenado.

---

## Vantagens da Merge Join

- Complexidade linear  $O(n+m)$  após ordenação
- Resultado naturalmente ordenado
- Eficiente para grandes volumes
- Aproveitamento de índices existentes

---

## Considerações de Desempenho

O custo total inclui a ordenação inicial das relações, que pode ser significativo, mas é compensado quando os dados já estão ordenados ou o resultado ordenado é necessário. A escolha entre merge join, hash join ou nested loop depende das características dos dados e do plano de execução completo.

O otimizador parte da **árvore inicial da consulta** e aplica **regras de equivalência algébrica** para gerar expressões equivalentes, explorando o espaço de planos possíveis.

Cada regra  $r$  aplicada a uma subexpressão  $e_i$  gera uma nova expressão  $E'$ . O plano de **menor custo estimado** é selecionado:

$$E^* = \operatorname{argmin}_{E \in EQ} \operatorname{custo\_estimado}(E)$$

⚠ A enumeração exaustiva é muito dispendiosa em espaço e tempo, exigindo abordagens como **programação dinâmica** para consultas complexas.

ENTRADA: consulta  $Q$  (árvore de álgebra relacional inicial)

PRÉ-CONDIÇÃO: conjunto de regras de equivalência  $R$  disponível

PASSO 1 — Inicialização

$EQ \leftarrow \{ \text{árvore\_inicial}(Q) \}$

PASSO 2 — Expansão iterativa

REPITA

$\text{nova\_expr} \leftarrow \text{false}$

PARA CADA expressão  $E$  em  $EQ$ :

PARA CADA subexpressão  $e_i$  em  $E$ :

PARA CADA regra  $r$  em  $R$ :

SE  $r$  é aplicável a  $e_i$ :

$E' \leftarrow \text{aplicar}(r, E, e_i)$

SE  $E' \notin EQ$ :

$EQ \leftarrow EQ \cup \{ E' \}$

$\text{nova\_expr} \leftarrow \text{true}$

ATÉ  $\text{nova\_expr} = \text{false}$

PASSO 3 — Seleção do plano ótimo

RETORNAR  $\operatorname{argmin}_{E \in EQ} \operatorname{custo\_estimado}(E)$

SAÍDA: expressão  $E^*$  de menor custo estimado

PÓS-CONDIÇÃO:  $E^*$  é semanticamente equivalente a  $Q$

Um plano de avaliação especifica completamente como uma consulta será executada, incluindo algoritmos, métodos de acesso e coordenação entre operações.

---

### **Algoritmos Específicos**

Para cada operação, o plano define o algoritmo exato: merge join, hash join, nested loop, entre outros.

---

### **Métodos de Acesso**

Define como os dados serão acessados: varredura linear, índice primário, índice secundário ou estruturas especializadas.

---

### **Coordenação de Execução**

Estabelece a ordem de execução, estratégias de pipeline, materialização de intermediários e uso de memória.

A seleção de um plano de avaliação ótimo requer considerar a interação entre diferentes técnicas e algoritmos. A escolha do algoritmo de menor custo para cada operação independentemente pode não gerar o melhor algoritmo geral.

---

## Exemplos de Interações

- **Merge Join:** pode ser mais caro que o hash join, mas produz saída ordenada, o que pode reduzir o custo de operações posteriores (ORDER BY, GROUP BY, outra junção).
- **Nested Loop Join:** Oferece oportunidades de pipelining, eliminando necessidade de materialização
- **Índices:** Uso estratégico pode beneficiar múltiplas operações na sequência

## Abordagens de Otimização

1. **Baseada em Custo:** Pesquisar todos os planos possíveis e escolher o de menor custo estimado
2. **Baseada em Heurística:** Usar regras práticas para escolher rapidamente um plano razoável

A otimização baseada em custo estima o custo de diferentes planos de execução e seleciona o de menor custo esperado.

---

### **Modelo de Custo**

Funções matemáticas que estimam tempo de CPU, operações de E/S e uso de memória para cada operação.

---

### **Estatísticas do Catálogo**

Informações sobre cardinalidade, distribuição de valores e características físicas do armazenamento.

---

### **Comparação de Planos**

Avaliação sistemática de alternativas, considerando combinações viáveis de algoritmos e ordens de operação.

Reutilizando resultados intermediários, o otimizador reduz o custo para  **$O(3^n)$**  em tempo e  **$O(2^n)$**  em espaço considerando árvores bushy (ou  **$O(n \cdot 2^n)$**  se restrito a árvores left-deep) — muito menos que enumerar todas as  **$(2(n-1))!/(n-1)!$**  ordens possíveis.

$$\text{bestPlan}(S) = \min_{S_1 \subset S} [\text{bestPlan}(S_1) + \text{bestPlan}(S \setminus S_1) + \text{joinCost}]$$

Usa **programação dinâmica** para encontrar o plano de junção de menor custo para um conjunto  $S$  de relações, com **memoização** para evitar recomputação.

**Fórmula de custo:**

$$cost(S) = \min_{\substack{S_1 \subset S \\ S_1 \neq \emptyset}} [cost(S_1) + cost(S \setminus S_1) + cost_{join}(S_1, S \setminus S_1)]$$

**Complexidade:**  $O(3^n)$  em tempo e  $O(2^n)$  subconjuntos memorizados vs.  $(2n-2)!/(n-1)!$  ordens possíveis.

ENTRADA:  $S$  (conjunto de relações a juntar)

PRÉ-CONDIÇÃO: `melhorplano[{r}]` inicializado

para cada relação base  $r \in S$

PASSO 1 — Verificar memoização

SE `melhorplano[S].custo`  $\neq \infty$ :

RETORNAR `melhorplano[S]`

PASSO 2 — Explorar bipartições de  $S$

PARA CADA subconjunto não vazio  $S_1 \subset S$  tal que  $S_1 \neq S$ :

$P_1 \leftarrow \text{acharMelhorPlano}(S_1)$

$P_2 \leftarrow \text{acharMelhorPlano}(S \setminus S_1)$

$A \leftarrow$  melhor algoritmo de junção para  $P_1 \quad P_2$

$c \leftarrow P_1.\text{custo} + P_2.\text{custo} + \text{custo}(A)$

SE  $c < \text{melhorplano}[S].\text{custo}$ :

`melhorplano[S].custo`  $\leftarrow c$

`melhorplano[S].plano`  $\leftarrow (P_1.\text{plano}; P_2.\text{plano}; \text{juntar via } A)$

PASSO 3 — Retornar solução ótima

RETORNAR `melhorplano[S]`

SAÍDA: `melhorplano[S]` — plano de menor custo para  $S$

PÓS-CONDIÇÃO: resultado é semanticamente equivalente à junção natural de todas as relações em  $S$

É fundamental balancear o tempo gasto otimizando com os ganhos esperados na execução da consulta.

---

### **Consultas Simples**

Tempo de otimização relativamente baixo, heurísticas frequentemente suficientes

---

### **Consultas Médias**

Equilíbrio entre otimização baseada em custo e heurística

---

### **Consultas Complexas**

Investimento em otimização é justificado pelos ganhos dramáticos na execução

Consultas frequentes (OLTP) podem ser otimizadas uma vez e reutilizadas, enquanto consultas ad-hoc complexas (OLAP) podem ter o tempo de otimização como fração significativa do tempo total.

A precisão das estimativas de cardinalidade é crucial para a qualidade do plano de execução escolhido pelo otimizador.

---

### **Cardinalidade Após Seleções**

Estimada usando fatores de seletividade baseados em histogramas e distribuições armazenadas no catálogo do sistema.

---

### **Cardinalidade Após Junções**

Calculada considerando chaves estrangeiras e assumindo independência ou dependência parcial entre atributos.

---

### **Impacto no Desempenho**

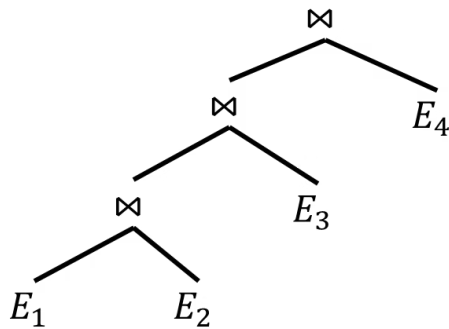
Resultados intermediários menores reduzem o custo de E/S e influenciam a escolha da ordem ideal de junção.



Estatísticas desatualizadas levam a planos subótimos — manutenção regular é essencial.

## Estrutura Esquerda Profunda

A entrada do lado direito de cada junção é sempre uma relação base, nunca um resultado intermediário — simplificando o espaço de busca do otimizador.



## Vantagens e Limitações

- **Pipelining:** O lado esquerdo alimenta continuamente a próxima junção
- **Espaço Reduzido:** Menos planos avaliados durante a otimização
- **Memória:** Apenas a relação direita precisa ser carregada por completo

O plano ótimo frequentemente pertence a esta classe, especialmente com nested loop joins.

O número de planos possíveis cresce exponencialmente, exigindo restrições estratégicas para tornar a otimização viável.

---

### **Restrição de Forma**

Considerar apenas árvores left-deep (em vez de todas as árvores bushy), reduzindo drasticamente o número de formatos avaliados.

---

### **Poda por Custo**

Descartar planos parciais cujo custo já excede o melhor plano completo encontrado até o momento.

---

### **Compromisso**

Balancear qualidade do plano final e tempo de otimização é essencial para aplicações práticas.

O "push down" de seleções é uma das regras de equivalência mais importantes na otimização de consultas.

| Regra Original                                                                                                                                                     | Transformação                                              | Regra Transformada                                  |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------|-----------------------------------------------------|
| $\sigma_{condicao}(R \bowtie S)$                                                                                                                                   | Decomposição em predicados que afetam apenas R ou apenas S | $(\sigma_{cond_R}(R)) \bowtie (\sigma_{cond_S}(S))$ |
| Seleção aplicada após a junção completa                                                                                                                            |                                                            | Seleções aplicadas antes da junção                  |
| <div><b>Push Down Selection:</b> Aplicar seleções cedo reduz resultados intermediários, diminuindo custos de E/S e processamento nas operações subsequentes.</div> |                                                            |                                                     |

Compreender os fatores que influenciam o custo da otimização é essencial para projetar estratégias eficazes e balancear tempo de otimização com qualidade do plano resultante.

---

### **Estratégias de Contenção**

- Timeout de otimização configurável
- Níveis de otimização (rápida, normal, exaustiva)
- Cache de planos para consultas repetidas

---

### **Benefícios do Investimento**

Para consultas complexas sobre grandes volumes, um bom plano pode ser ordens de magnitude mais rápido que um plano ingênuo.

Uma "ordem interessante" é uma ordenação dos dados que pode beneficiar operações subsequentes, e o otimizador deve considerá-la mesmo que implique custo inicial maior.

---

## **Identificação**

Detectar atributos de ORDER BY, GROUP BY e junções que se beneficiam de dados ordenados.

---

## **Avaliação**

Comparar o custo de produzir a ordem interessante versus realizar uma ordenação explícita posterior.

---

## **Propagação**

Rastrear quais operações preservam, destroem ou criam ordens interessantes.

---

## **Decisão**

Escolher o plano que considera os benefícios globais da ordenação em toda a consulta.

Quando a otimização baseada em custo é **proibitivamente cara**, heurísticas aplicam regras de transformação em ordem fixa que produzem planos eficientes sem enumerar todo o espaço de busca:

$$H : Q \rightarrow Q'$$

Heurísticas trocam **otimalidade garantida** por **velocidade de otimização**; sistemas modernos combinam heurísticas com otimização parcial por custo.

ENTRADA:  $Q$  (árvore de álgebra relacional)

PRÉ-CONDIÇÃO:  $Q$  pode conter produtos cartesianos e seleções não empurradas

PASSO 1 — Eliminar produtos cartesianos

se  $\sigma_{\theta}(R \times S)$  e  $\theta$  envolve  $R$  e  $S$ :

substituir por  $R \bowtie_{\theta} S$

PASSO 2 — Push-down de seleções ( $\sigma$ )

mover cada  $\sigma_{\theta}$  (bottom-up) para o nó

mais próximo das relações base de  $\theta$

PASSO 3 — Push-down de projeções ( $\pi$ )

inserir  $\pi$  intermediárias abaixo de junções,

eliminando atributos não usados adiante

PASSO 4 — Reordenar por seletividade

ordenar  $\sigma$  e  $\pi$  da mais para a menos

restritiva (menor cardinalidade primeiro)

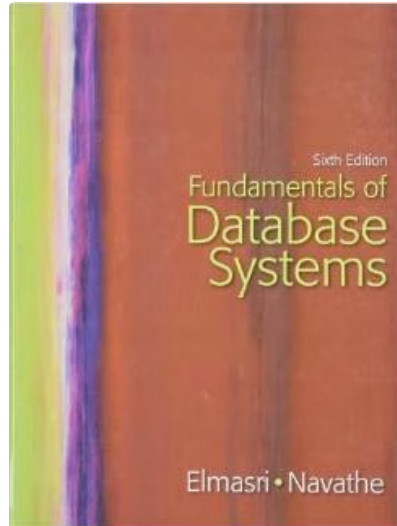
PASSO 5 — Identificar pipelining

marcar sub-árvores sem materialização

obrigatória para execução em pipeline

SAÍDA:  $Q' \equiv Q$ , custo esperado  $\leq \text{custo}(Q)$

## Referências

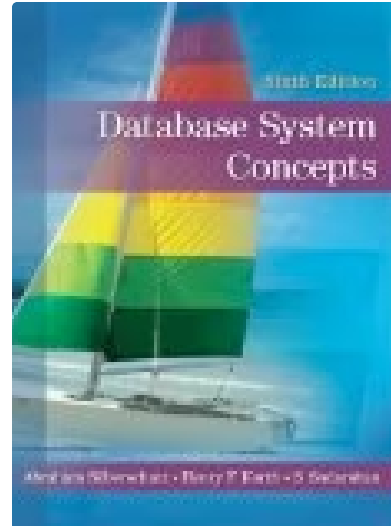


---

**Elmasri & Navathe**

**Fundamentals of Database Systems**

Pearson, 2016

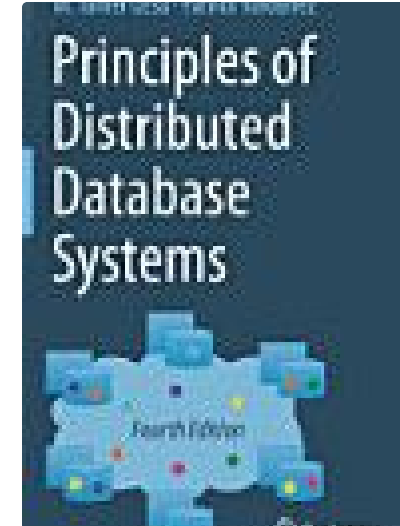


---

**Korth, Sudarshan & Silberschatz**

**Database System Concepts**

McGraw-Hill, 2019



---

**Özsu & Valduriez**

**Principles of Distributed Database Systems**

Springer Nature, 2019